ORF307_HW5_soln

ORF307 Homework 5 Solutions

Due: Friday, October 31, 2025 11:59 pm ET

Q1 Finding a direction in simplex

Let $P = \{x \in \mathbf{R}^3 \mid 2x_1 + 3x_2 + x_3 = 1, x \geq 0\}$ and consider the vector $x = (0, 0, 1)$. Let the cost vector be $c = (1, -1, 2)$. Find the set of basic directions at x that improve the cost. Assume it is a minimization problem, so we want to decrease the cost.

In this problem, we have A as a row vector $A = (2, 3, 1)$. We need $Ad = 0$ to maintain feasibility. The current basis is just $\{3\}$, the third index. So there will be 2 possible basic directions. The first basic direction is $d_1 = (1, 0, -2)$ and the second basic direction is $d_2 = (0, 1, -3)$. We see that both directions improve the cost since $c^T d_1 < 0$ and $c^T d_2 < 0$

Q2 Extreme points and basic feasible solutions

Let $P = \{x \in \mathbf{R}^5 \mid Ax = b, x \geq 0\}$ where

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & -2 \\ 0 & -1 & 1 & -1 & 0 \\ 2 & 0 & 1 & 0 & -1 \end{bmatrix}, \quad \text{and} \quad b = (1, 1, 1).$$

(a) Given the following 3 vectors

i. $\hat{x} = (0, 0, 1, 0, 0)$

ii. $\hat{x} = (0, 0, 1, 1, 1)$

iii. $\hat{x} = (0, 0, 0, -1, -1)$

list which ones are in P, which ones are basic solutions, and which ones are degenerate.

i. feasible, basic, degenerate

ii. infeasible, not basic

iii. infeasible, basic

(b) If they are basic feasible solutions are they extreme points? If yes, give a vector c for which $\hat{x}$ is the unique solution of

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & Ax = b \\ & x \geq 0 \end{array}$$

Yes, (i) is a basic feasible solution and it is an extreme point. We can pick $c = (1, 1, 0, 1, 1)$. The objective value must be non-negative. And $(0, 0, 1, 0, 0)$ is the only way to achieve an objective value of zero.

[2]:
```
# to double-check with cvxpy
import numpy as np
import cvxpy as cp
x = cp.Variable(5)
A = np.array([[0, 1, 1, 1, -2],
              [0, -1, 1, -1, 0],
              [2, 0, 1, 0, -1]])
b = np.array([1, 1, 1])
c = np.array([1, 1, 0, 1, 1])
constraints = [A @ x == b, x >= 0]
prob = cp.Problem(cp.Minimize(c @ x), constraints)
prob.solve()
print(x.value)
```

```
[-0.  0.  1. -0. -0.]
```

Q3 Simplex iterations

Solve the following optimization problem using the simplex method. At each iteration, record the indices in the basis, x, the reduced costs $\bar{c}$, the new feasible direction d, and step length θ^*. Any method to come up with a starting basic feasible solution is ok.

$$\begin{array}{ll} \text{minimize} & 8x_1 + 8x_2 + 5x_3 + 9x_4 \\ \text{subject to} & 2x_1 + x_2 + x_3 + 3x_4 = 5 \\ & x_1 + 3x_2 + x_3 + 2x_4 = 3 \\ & x \geq 0 \end{array}$$

For this problem, we can run the simplex code given and record or print the important pieces of information along the way. We can use $(2, 0, 1, 0)$ as an initial basic feasible solution. We see that we get the following. We've added the verbose flag to the code provided.

[4]:
```
problem = {'A': np.array([[2, 1, 1, 3], [1, 3, 1, 2]]),
           'b': np.array([5, 3]),
           'c': np.array([8, 8, 5, 9])}
x0 = np.array([2, 0, 1, 0])
B0 = np.array([0, 2])
print('init B', B0)
x, B = simplex_algorithm(x0, B0, problem, verbose=True)
```

```
init B [0 2]
x [2 0 1 0]
c_bar [ 0. -1.  0. -4.]
d [ 2.  1. -5.  0.]
theta 0.2
B_next [0 1]
x [2.4 0.2 0.  0. ]
c_bar [ 0.   0.   0.2 -3.8]
d [-1.4 -0.2  0.   1. ]
theta 1.0
B_next [0 3]
x [1. 0. 0. 1.]
c_bar [ 0. 19.  4.  0.]
Optimal solution found!
```

The optimal solution is $x = (1, 0, 0, 1)$

[3]:
```
'''
This code is provided to help with questions 3 and 4
'''

import numpy as np
import numpy.linalg as la

def simplex_iteration(x, B, problem, verbose=False):
    """Perform one simplex iteration given
    - basic feasible solution x
    - basis B

    It returns new x, new basis and termination flag (true/false)
    """
    A, b, c = problem['A'], problem['b'], problem['c']
    m, n = A.shape
    A_B = A[:, B]

    # Compute reduced cost vector
    p = la.solve(A_B.T, c[B])
    c_bar = c - A.T @ p

    if verbose:
        print('c_bar', c_bar)

    # Check optimality
    if np.all(c_bar >= 0):
        print("Optimal solution found!")
        return x, B, True

    # Choose j such that c_bar < 0 (first one)
```

```
    j = np.where(c_bar < 0)[0][0]

    # Compute search direction d
    d = np.zeros(n)
    d[j] = 1
    d[B] = la.solve(A_B, -A[:, j])

    if verbose:
        print('d', d)

    # Check for unboundedness
    if np.all(d >= 0):
        print("Unbounded problem!")
        return None, None, True

    # Compute step length theta
    d_i = np.where(d[B] < 0)[0]
    theta = np.min(- x[B[d_i]] / d[B[d_i]])
    if verbose:
        print('theta', theta)
    i = B[d_i[np.argmin(- x[B[d_i]] / d[B[d_i]])]]

    # Compute next point
    x_next = x + theta * d

    # Compute next basis
    B_next = B
    B_next[np.where(B == i)[0]] = j
    if verbose:
        print('B_next', B)

    return x_next, B_next, False

def simplex_algorithm(x, B, problem, max_iter=1000, verbose=False):
    """Run simplex algorithm"""

    for k in range(max_iter):
        if verbose:
            print('x', x)
        x, B, end = simplex_iteration(x, B, problem, verbose=verbose)

        if end:
            break
    return x, B
```

[5]:
```
#verification in cvxpy
import cvxpy as cp
```

```
x = cp.Variable(4)
constraints = [problem['A']@x == problem['b']]
constraints +=[x >= 0]
objective = cp.Minimize(problem["c"]@x)
prob = cp.Problem(objective, constraints)
prob.solve()
x.value
```

[5]:
```
array([ 1., -0.,  0.,  1.])
```

[ ]:
```

```